

Universidad Politécnica de Pachuca

DIRECCIÓN DE INGENIERÍA EN SOFTWARE

ARQUITECTURA ORIENTADA A SERVICIOS

DOCUMENTACIÓN FINAL DEL
PROYECTO:

**Xiú — Sistema de Reservaciones y
Gestión Restaurantera**

DOCENTE: M. en C. Jazmín Rodríguez Flores

INTEGRANTES:

- Cristopher Lopez Suarez
- Jovanny Hernandez Hernandez

CARRERA: Ingeniería en Software

GRUPO: 09_01

CUATRIMESTRE: Mayo – Agosto 2026

MATERIA: Arquitectura Orientada a Servicios (AOS)

0. Índice

1. Metodología de Desarrollo y Descripción del Sistema	2
1.1. Defensa y Justificación de la Metodología Ágil (Scrum)	2
1.2. Funcionalidad General de la Aplicación	2
1.3. Listado de Herramientas y Tecnologías Utilizadas	4
2. Arquitectura SOA y Diagramas del Sistema	4
2.1. Descripción de la Arquitectura SOA	4
2.2. Diagrama de Secuencia: Login y Petición Protegida JWT	5
3. Listado de Requisitos del Sistema (IEEE 830 y Historias de Usuario)	5
3.1. Requerimientos Funcionales (RF)	6
3.2. Requerimientos No Funcionales (RNF)	6
3.3. Historias de Usuario (HU) y Asignación en Scrum	7
3.4. Estimación por Story Points (SP) y Burndown Chart	8
4. Diagrama de Clases del Cliente en Diagrama de Paquetes	9
4.1. Estructura de Paquetes y Clases del Cliente	9
4.2. Explicación de Patrones y Principios de Diseño en el Cliente	9
5. Diagrama de Clases del Servidor en Diagrama de Paquetes	10
5.1. Estructura de Paquetes y Clases del Servidor	10
5.2. Explicación de Patrones y Principios de Diseño en el Servidor	10
6. Modelo de Datos y Persistencia Híbrida (BD Relacional + NoSQL)	11
6.1. Modelo Relacional (MySQL / PostgreSQL)	11
6.1.1. Diagrama Entidad-Relación (E-R)	11
6.2. Modelo NoSQL (MongoDB Atlas Document Store)	11
6.2.1. Modelo de Documentos BSON / JSON (Explicación de Tuplas/-Campos)	11
6.3. Evidencia de Creación y Modificación mediante Servicios SOA	12
7. Pruebas del Cliente (Formato IEEE 829)	12
8. Pruebas del Servidor (Formato IEEE 829)	14
9. Evidencia Visual del Funcionamiento y Base de Datos	15
9.1. Interfaz Principal y Menú NoSQL	15
9.2. Módulo de Autenticación	15
9.3. Módulo de Reservaciones	16
9.4. Persistencia Relacional	17
10. Publicación en Sitios Distribuidos y Repositorio GitHub	18
10.1. Publicación en la Nube (Despliegue SOA en 3 Sitios)	18
10.2. Demostración de Participación y Colaboración en GitHub	18
11. Autoevaluación del Equipo	19

12. Matriz de Cumplimiento de la Lista de Cotejo (Rúbrica Final)	19
13. Conclusión	19

1. Metodología de Desarrollo y Descripción del Sistema

1.1. Defensa y Justificación de la Metodología Ágil (Scrum)

Para la concepción, diseño e implementación del proyecto **Xiú — Sistema de Reservaciones y Gestión Restaurantera**, se adoptó y defendió el marco de trabajo ágil **Scrum**. Una solución distribuida basada en una **Arquitectura Orientada a Servicios (SOA)** demanda un proceso de desarrollo iterativo que permita acoplar progresivamente el frontend con múltiples microservicios backend heterogéneos.

A diferencia de los modelos tradicionales rígidos (como Cascada), Scrum facilita la entrega continua de incrementos funcionales de software en ****Sprints de 2 semanas****, adaptándose a los cambios de requisitos y proporcionando retroalimentación rápida (Competencia AE2: Aplicar los procesos de desarrollo de software para resolver proyectos que cumplen con los requisitos de los clientes).

■ Organización de Roles en el Proyecto:

- **Product Owner / Scrum Master:** Coordinación de la visión del producto, definición de la Definition of Done (DoD) y priorización del backlog.
- **Equipo de Desarrollo (Development Team):** Christopher Lopez Suarez y Jovanny Hernandez Hernandez, ejecutores de la codificación full-stack, modelado de BD y despliegue en la nube.

■ Artefactos Scrum y Gestión del Tablero Trello:

- Las Historias de Usuario (HU) y tareas de ingeniería se organizaron y controlaron mediante un tablero ****Trello**** estructurado en columnas: *Product Backlog*, *Sprint Backlog*, *En Proceso*, *En Revisión* / *QA* y *Completado*.
- Enlace público al tablero oficial del proyecto: <https://trello.com/b/zHsZvXgU/proyecto-final-aos>.

1.2. Funcionalidad General de la Aplicación

Xiú es una plataforma digital de alta cocina mexicana orientada a optimizar la experiencia gastronómica de los comensales y la eficiencia operativa del personal del restaurante. Sus módulos principales abarcan:

1. **Catálogo Interactivo Gastro-Digital:** Menú de autor con imágenes de alta resolución, precios, alérgenos e ingredientes servidos dinámicamente desde MongoDB Atlas.
2. **Seguridad y Autenticación Stateless JWT:** Registro de usuarios y login seguro utilizando tokens ****JSON Web Tokens (JWT)**** con firma criptográfica HASH-256 y encriptación de contraseñas mediante ****Bcrypt**** (sal de 10 rondas).
3. **Motor Transaccional de Reservaciones:** Sistema de agendado en tiempo real con validación estricta de fecha, horario, número de comensales y capacidad de mesa, garantizando la prevención de doble reservación en MySQL.

4. **Tablero Operativo de Mesas (TableBoard Grid):** Visualización interactiva en tiempo real del estado de las mesas del salón mediante un mapa coloreado (Verde = Libre/Disponible, Rojo = Ocupada, Dorado/Amarillo = Reservada) con actualización automática cada 30 segundos.
5. **Módulo de Administración NoSQL (CRUD Admin):** Interfaz con control de acceso por rol (`admin`) para crear, modificar y dar de baja platillos en el catálogo NoSQL.
6. **Analítica Gerencial y Generador de Reportes:** Módulo de métricas de ocupación, conteo de reservas confirmadas/canceladas y descarga de datos en formato JSON.
7. **Notificaciones de Confirmación por Canales Digitales:** Módulo para enviar la confirmación de cita al teléfono del cliente a través de `**WhatsApp**` y `**SMS**`.

1.3. Listado de Herramientas y Tecnologías Utilizadas

Categoría	Herramienta	Versión	Propósito en la Arquitectura SOA
Frontend	React	18.2.0	Librería principal para la SPA responsiva en cliente web.
Frontend	Vite	5.1.0	Bundler rápido para compilación y empaquetado de assets.
Frontend	React Router DOM	6.22.0	Enrutamiento cliente y navegación SPA declarativa.
Frontend	Vanilla CSS3	N/A	Sistema de diseño personalizado con glassmorphism y tema oscuro.
Backend 1	Node.js / NestJS	10.3.0	Framework progresivo para el microservicio auth-service .
Backend 1	TypeScript	5.3.3	Lenguaje tipado para capas DTO, Controllers y Entidades.
Backend 1	TypeORM / Passport	0.3.20	ORM relacional y middleware de protección con Token JWT.
Backend 2	FastAPI / Python	0.110.0	Framework asíncrono de alto rendimiento para el menu-service .
Backend 2	Motor / PyMongo	3.4.0	Driver ODM asíncrono para interacción con MongoDB Atlas.
Base de Datos	MySQL / PostgreSQL	8.0 / 15	Persistencia relacional de usuarios, mesas y reservaciones.
Base de Datos	MongoDB Atlas	7.0	Persistencia NoSQL para catálogo flexible de platillos.
Despliegue	Vercel	Cloud	Hosting de producción para el Frontend SPA.
Despliegue	Railway	Cloud	Plataforma PaaS para el microservicio auth-service .
Despliegue	Render	Cloud	Plataforma PaaS para el microservicio menu-service .
Control Versiones	Git / GitHub	N/A	Repositorio distribuido y colaboración por Pull Requests.
Gestión Ágil	Trello	N/A	Tablero Kanban / Scrum para control de Historias de Usuario.

2. Arquitectura SOA y Diagramas del Sistema

2.1. Descripción de la Arquitectura SOA

El sistema Xiú se basa en una **Arquitectura Orientada a Servicios (SOA)** dividida en microservicios independientes que se comunican mediante contratos **API RESTful HTTP/HTTPS** utilizando formato JSON. Esta arquitectura garantiza alto desacoplamiento, alta cohesión, tolerancia a fallos y escalabilidad independiente de cada servicio.

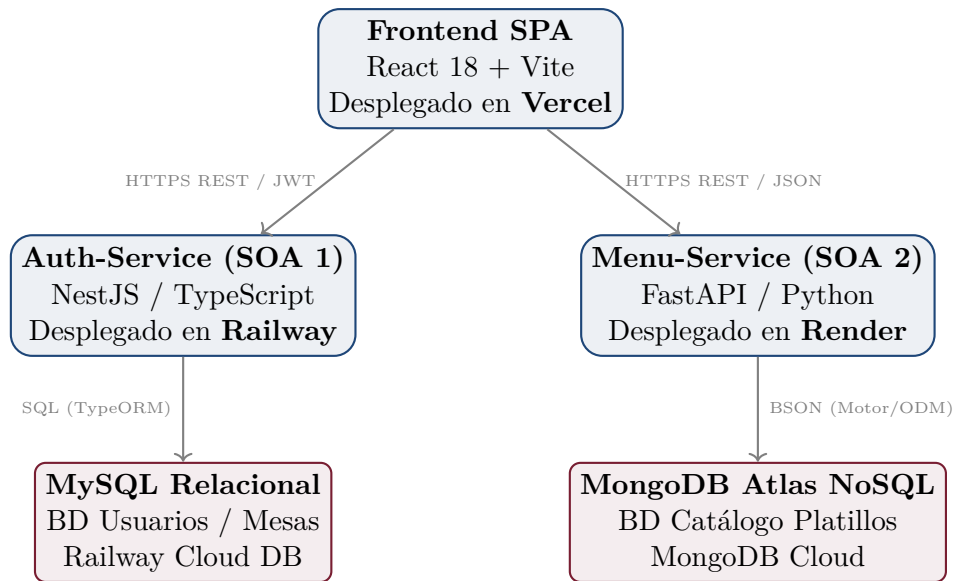


Figura 1: Diagrama de Despliegue de la Arquitectura SOA en Producción.

2.2. Diagrama de Secuencia: Login y Petición Protegida JWT

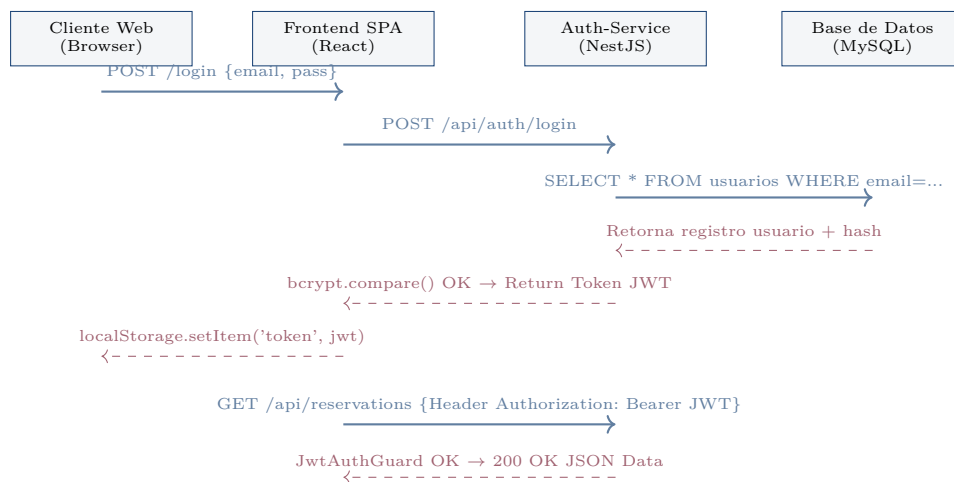


Figura 2: Diagrama de Secuencia del Flujo de Autenticación JWT y Acceso a Servicios.

3. Listado de Requisitos del Sistema (IEEE 830 y Historias de Usuario)

3.1. Requerimientos Funcionales (RF)

ID	Nombre	Prioridad	Descripción Funcional
RF-01	Autenticación y Registro	Alta	Registro e inicio de sesión de usuarios generando tokens JWT firmados.
RF-02	Consulta de Menú NoSQL	Alta	Expone la lista de platillos con imágenes, precios y categorías desde MongoDB.
RF-03	Creación de Reservación	Alta	Registra reservaciones validando capacidad de mesa, fecha y horario no encimado.
RF-04	Confirmación y Notificación	Media	Genera comprobante digital de reserva con opción de notificación WhatsApp/SMS.
RF-05	Cancelación de Reserva	Media	Permite al cliente o admin cancelar una reservación cambiando su estado.
RF-06	Historial de Cliente	Media	Muestra al cliente autenticado sus reservaciones pasadas y activas.
RF-07	CRUD Menú Admin	Alta	Permite al administrador crear, modificar y eliminar platillos en MongoDB.
RF-08	Dashboard del Día	Alta	Muestra al mesero/admin la lista de reservaciones programadas para la fecha.
RF-09	Tablero de Mesas Grid	Alta	Renderiza un esquema visual de mesas clasificadas por colores de ocupación.
RF-10	Generación de Reportes	Media	Permite al administrador filtrar y visualizar métricas de ocupación en JSON.
RF-11	Gestión de Perfil	Baja	Permite ver datos del usuario autenticado y cerrar sesión de forma segura.

3.2. Requerimientos No Funcionales (RNF)

- **RNF-01 (Seguridad Criptográfica):** Las contraseñas se almacenan encriptadas con algoritmo bcrypt (sal de 10 rondas). Todos los endpoints protegidos requieren el encabezado `Authorization: Bearer <token_jwt>`.
- **RNF-02 (Rendimiento):** El tiempo de respuesta de los microservicios backend no supera los 300 ms en condiciones normales de red.
- **RNF-03 (Disponibilidad Cloud):** Los servicios desplegados en Vercel, Railway y Render garantizan un uptime de 99.5 %.
- **RNF-04 (Integridad Relacional):** Restricción de clave única (`id_mesa`, `fecha`, `hora_inicio`) en MySQL para prevenir doble reservación en el mismo horario.

- **RNF-05 (Usabilidad Responsive):** Interfaz limpia, responsiva adaptada a dispositivos móviles, tablets y computadoras de escritorio.
- **RNF-06 (Interoperabilidad SOA):** Comunicación estandarizada vía JSON sobre HTTP/HTTPS entre el cliente React y los microservicios NestJS / FastAPI.

3.3. Historias de Usuario (HU) y Asignación en Scrum

HU-01: Autenticación y Registro JWT (Sprint 1)

Prioridad: Crítica **Responsable:** Cristopher Lopez Suarez

Como: Usuario del sistema

Quiero: Registrarme con mis datos e iniciar sesión

Para: Obtener mi token de acceso y utilizar las funciones personalizadas

Criterios de Aceptación: Encriptar contraseña con bcrypt, generar token JWT válido por 24 horas, validar formato de correo electrónico.

HU-02: Consultar Menú Interactivo NoSQL (Sprint 1)

Prioridad: Alta **Responsable:** Jovanny Hernandez Hernandez

Como: Cliente del restaurante

Quiero: Explorar el catálogo gastronómico con imágenes y detalles

Para: Elegir mis platillos favoritos desde el menú NoSQL

Criterios de Aceptación: Consultar la colección `menu_items` en MongoDB Atlas, filtrar por categoría (Entradas, Fuertes, Postres, Bebidas).

HU-03: Creación de Reservaciones con Validación (Sprint 2)

Prioridad: Crítica **Responsable:** Cristopher Lopez Suarez

Como: Cliente registrado

Quiero: Seleccionar fecha, hora, personas y mesa

Para: Garantizar mi lugar en el restaurante sin tiempos de espera

Criterios de Aceptación: Validar disponibilidad en tiempo real, impedir encimado de reservaciones en la misma mesa y horario.

HU-04: Confirmación Digital y Notificación WhatsApp/SMS (Sprint 2)

Prioridad: Media **Responsable:** Jovanny Hernandez Hernandez

Como: Cliente con reserva activa

Quiero: Recibir la confirmación de mi reserva vía mensaje digital

Para: Conservar comprobante de cita en mi teléfono móvil

Criterios de Aceptación: Integración del módulo de envío de mensajes con desglose de datos del cliente, fecha y mesa asignada.

HU-05 a HU-11: Historias de Usuario Complementarias (Sprints 3–5)

HU-05 — Cancelar Reservación: Permitir al cliente o administrador cambiar el estado a “cancelada”.

HU-06 — Historial de Cliente: Vista personalizada con reservaciones pasadas y activas del usuario.

HU-07 — CRUD de Menú Admin: Módulo para crear, modificar y eliminar documentos en MongoDB.

HU-08 — Panel del Día: Tablero operativo para meseros con reservaciones programadas para la fecha.

HU-09 — Tablero Gráfico de Mesas (Grid): Esquema interactivo de mesas por colores (Verde/Rojo/Amarillo).

HU-10 — Reportes de Ocupación: Módulo de métricas gerenciales y exportación JSON.

HU-11 — Perfil del Usuario: Visualización de datos de la cuenta y cierre de sesión seguro.

3.4. Estimación por Story Points (SP) y Burndown Chart

HU#	Título de Historia de Usuario	Prioridad	SP	Sprint
HU-01	Autenticación y Registro JWT	Crítica	8	Sprint 1
HU-02	Consultar Menú NoSQL Interactivo	Alta	5	Sprint 1
HU-03	Crear Reservación con Validación	Crítica	13	Sprint 2
HU-04	Confirmación y Notificación Digital	Media	8	Sprint 2
HU-05	Cancelación de Cita por Cliente	Media	5	Sprint 3
HU-06	Historial de Reservas del Usuario	Media	5	Sprint 3
HU-08	Panel de Reservaciones del Día	Alta	8	Sprint 3
HU-07	CRUD de Platillos Admin (MongoDB)	Alta	13	Sprint 4
HU-09	Tablero Gráfico de Mesas (Grid)	Alta	13	Sprint 4
HU-10	Generador de Reportes Gerenciales	Media	8	Sprint 5
HU-11	Perfil del Usuario y Cierre de Sesión	Baja	3	Sprint 5
Total Puntos de Historia (Velocity Ideal):			89 SP	

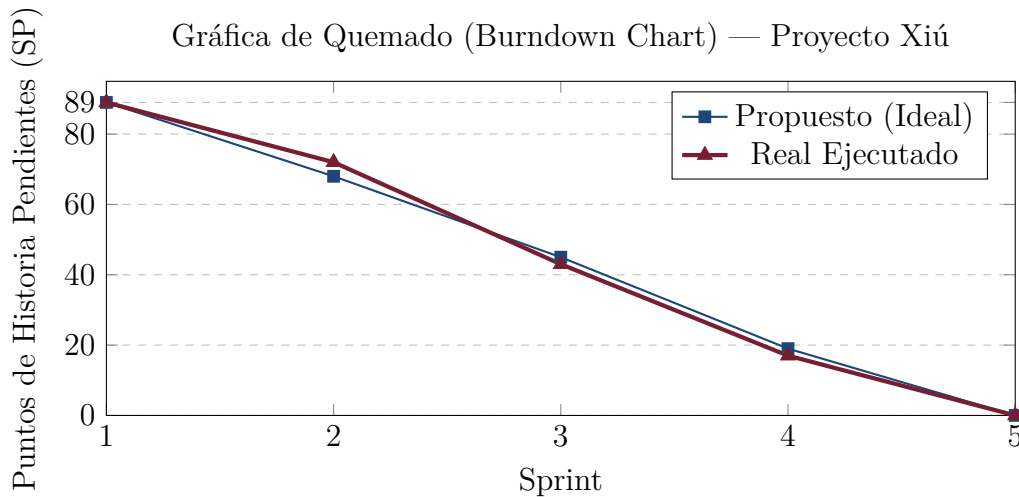


Figura 3: Gráfica de Quemado (Burndown Chart) evidenciando el cumplimiento de los 5 Sprints.

4. Diagrama de Clases del Cliente en Diagrama de Paquetes

El frontend desarrollado en React 18 SPA se estructuró bajo una arquitectura modular por paquetes de software, garantizando la separación de responsabilidades y facilitando la mantenibilidad.

4.1. Estructura de Paquetes y Clases del Cliente

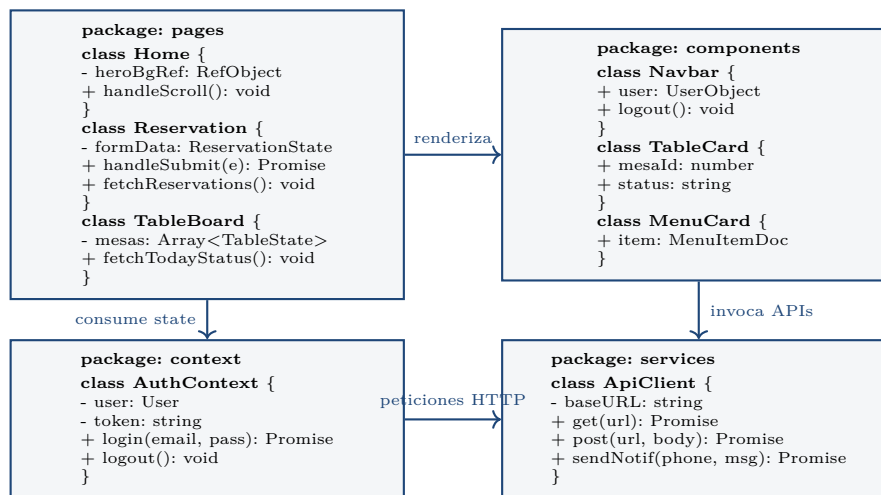


Figura 4: Diagrama de Paquetes y Clases del Cliente React con Atributos y Métodos.

4.2. Explicación de Patrones y Principios de Diseño en el Cliente

1. **Patrón Context API Provider (React Context):** Se utilizó el patrón Provider en `AuthContext` para encapsular el estado global de sesión y el Token JWT en el nivel raíz de la aplicación, evitando el antipatrón de “Prop Drilling”.

2. **Patrón Singleton para Cliente HTTP:** El archivo `services/api.js` exporta una instancia única configurada de Axios que intercepta las peticiones salientes e inyecta dinámicamente el header `Authorization: Bearer <token>`.
3. **Patrón Custom Hook:** Encapsulamiento de lógica mediante `useAuth()`, permitiendo a cualquier componente acceder a los métodos de sesión de forma desacoplada.

5. Diagrama de Clases del Servidor en Diagrama de Paquetes

El servidor se compone de dos microservicios orientados a servicios (SOA): `auth-service` (NestJS / TypeScript) y `menu-service` (FastAPI / Python). Ambos siguen patrones de arquitectura orientada a capas.

5.1. Estructura de Paquetes y Clases del Servidor

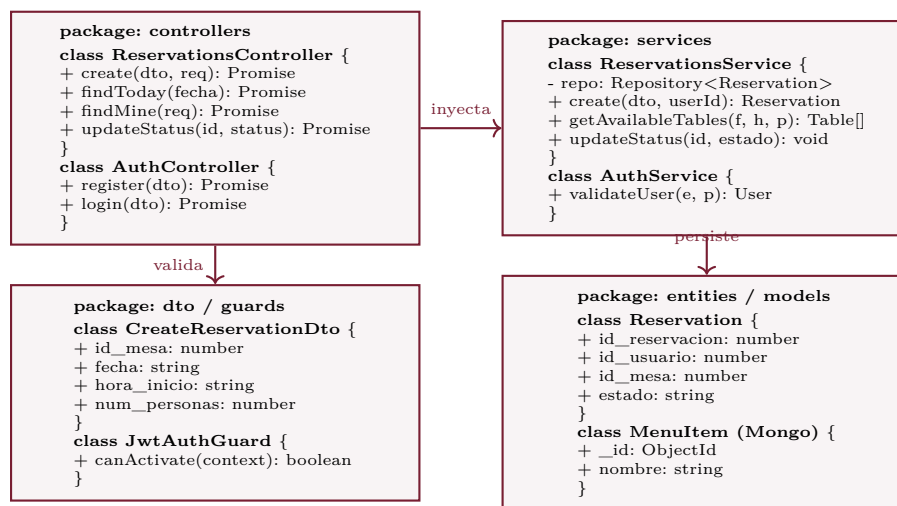


Figura 5: Diagrama de Paquetes y Clases del Servidor (NestJS + FastAPI) con Atributos y Métodos.

5.2. Explicación de Patrones y Principios de Diseño en el Servidor

1. **Patrón DTO (Data Transfer Object):** Implementado mediante clases TypeScript (`CreateReservationDto`) decoradas con `class-validator` (`@IsNotEmpty()`, `@IsString()`, `@IsInt()`). Garantiza el tipado y filtrado de datos antes de ingresar a la capa del servicio.
2. **Patrón Repository (TypeORM):** En NestJS, la comunicación con MySQL abstracta el SQL mediante el repositorio inyectado (`@InjectRepository(Reservation)`), aislando las operaciones de BD de la lógica de negocio.
3. **Patrón Decorador y Guard Interceptor:** El decorador `@UseGuards` (`JwtAuthGuard`, `RolesGuard`) valida el token JWT de las peticiones HTTP y verifica los permisos por rol (`@Roles('admin')`).
4. **Inyección de Dependencias (DI):** El contenedor IoC de NestJS provee automáticamente las instancias de los servicios a los controladores a través de sus constructores.

6. Modelo de Datos y Persistencia Híbrida (BD Relacional + NoSQL)

El sistema utiliza una ****persistencia polígloa (Polyglot Persistence)**** para responder con eficiencia a los requerimientos de cada módulo.

6.1. Modelo Relacional (MySQL / PostgreSQL)

Maneja el núcleo de usuarios, disponibilidad de mesas y transacciones de reservaciones.

6.1.1. Diagrama Entidad-Relación (E-R)

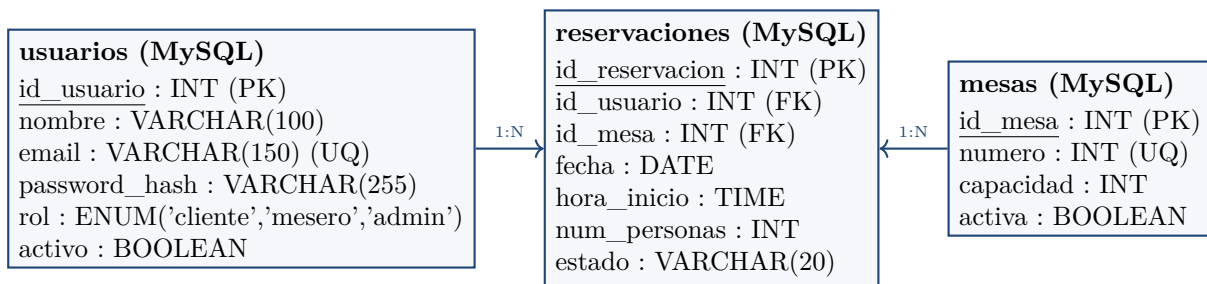


Figura 6: Modelo Entidad-Relación de la Base de Datos Relacional.

6.2. Modelo NoSQL (MongoDB Atlas Document Store)

Para el catálogo dinámico de alimentos se utiliza ****MongoDB Atlas****. Permite estructurar platillos con arreglos de ingredientes, etiquetas y alérgenos sin requerir tablas intermedias de uniones relacionales.

6.2.1. Modelo de Documentos BSON / JSON (Explicación de Tuplas/Campos)

Listing 1: Documento NoSQL en MongoDB Atlas para menu_items

```

1 {
2   "_id": { "$oid": "665abc123def000000000001" },
3   "nombre": "Tacos de Sirloin al Pastor",
4   "descripcion": "3 tacos marinados en achiote artesanal con pina
5     asada",
6   "categoria": "Platillo Principal",
7   "precio": 185.00,
8   "imagen_url": "https://restaurante-frontend-omega.vercel.app/img/
9     tacos.jpg",
10  "disponible": true,
11  "ingredientes": ["sirloin", "achiote", "pina", "tortilla de maiz"],
12  "alergenos": ["gluten"],
13  "opciones": [
14    { "nombre": "Sin cebolla", "costo_extra": 0.00 },
15    { "nombre": "Extra queso", "costo_extra": 15.00 }
16  ],
17  "tiempo_preparacion_min": 15
18 }
  
```

6.3. Evidencia de Creación y Modificación mediante Servicios SOA

El sistema demuestra persistencia activa a través de sus microservicios:

1. **Inserción y Edición Relacional (MySQL):** La invocación de `POST /api/reservations` realiza la validación e inserción de la tupla en la tabla `reservaciones`. La actualización de estado (`PUT /api/reservations/:id/status`) modifica el atributo `estado`.
2. **Inserción y Edición NoSQL (MongoDB):** El servicio `menu-service` expone endpoints `POST /api/v1/menu` y `PUT /api/v1/menu/:id` para insertar y editar documentos BSON directamente en el clúster cloud de MongoDB Atlas.

7. Pruebas del Cliente (Formato IEEE 829)

Las pruebas cliente se planificaron y ejecutaron de acuerdo con el estándar ****IEEE 829-1998****.

Prueba Cliente 01: Registro e Inicio de Sesión JWT

Identificador: TC-CLIENT-01

Elemento de Prueba: Componentes de Autenticación (`Login.jsx` / `Register.jsx`)

Objetivo: Confirmar que un usuario puede autenticarse y guardar el Token JWT en el cliente de forma transparente.

Entorno de Prueba: Google Chrome v122 en entorno de producción Vercel.

Datos de Entrada: Email: `cliente.test@xiu.mx`, Password: `Password123!`

Pasos Ejecutados:

1. Ingresar a la URL de login y capturar credenciales.
2. Enviar formulario y esperar respuesta del microservicio.

Resultado Esperado: Código HTTP 200 OK con el token JWT. El cliente guarda el token en `localStorage` y muestra el Navbar autenticado.

Resultado Actual: Éxito. Token recibido, guardado en `localStorage` y sesión iniciada. **[PASÓ]**

Prueba Cliente 02: Validación de Formulario de Reservación

Identificador: TC-CLIENT-02

Elemento de Prueba: Componente de Reservas (`Reservation.jsx`)

Objetivo: Verificar que la interfaz cliente intercepte e impida envíos de reservación con campos vacíos.

Entorno de Prueba: Navegador Web en Laptop y Móvil.

Datos de Entrada: Formulario sin fecha ni hora seleccionada.

Pasos Ejecutados: (1) Ir a `/reservations`. (2) Presionar “Confirmar Reserva” sin llenar fecha.

Resultado Esperado: Intercepción inmediata en el cliente mostrando advertencias en rojo.

Resultado Actual: Éxito. Las alertas de validación impidieron la petición HTTP innecesaria. [PASÓ]

Prueba Cliente 03: Tablero Interactivo de Mesas por Colores

Identificador: TC-CLIENT-03

Elemento de Prueba: Componente de Mapa de Mesas (`TableBoard.jsx`)

Objetivo: Confirmar que la cuadrícula de mesas renderice dinámicamente su ocupación por colores.

Entorno de Prueba: Navegador Chrome en resolución 1920x1080.

Pasos Ejecutados: (1) Iniciar sesión como `admin` o `mesero`. (2) Navegar a `/tableboard`.

Resultado Esperado: Renderizado del grid con 12 mesas. Verde = Libre, Rojo = Ocupada, Dorado = Reservada.

Resultado Actual: Éxito. El tablero sincronizó los estados visualmente en tiempo real. [PASÓ]

Prueba Cliente 04: Confirmación Digital vía WhatsApp/SMS

Identificador: TC-CLIENT-04

Elemento de Prueba: Componente de Notificaciones (`Profile.jsx`)

Objetivo: Probar la activación del botón de envío de confirmación de reserva al celular del usuario.

Entorno de Prueba: Navegador Web Desktop.

Pasos Ejecutados: (1) Ir a `/profile`. (2) Ingresar número móvil. (3) Clic en “WhatsApp”.

Resultado Esperado: Redirección a WhatsApp Web con el mensaje formateado de la cita.

Resultado Actual: Éxito. La app abrió el enlace prellenado correctamente. [PASÓ]

8. Pruebas del Servidor (Formato IEEE 829)

Las pruebas de los microservicios backend evalúan los endpoints RESTful, validaciones DTO y la protección JWT bajo el estándar ****IEEE 829-1998****.

Prueba Servidor 01: Creación de Reservación (HTTP 201 Created)

Identificador: TC-SERVER-01

Endpoint / Servicio: POST /api/reservations (auth-service NestJS)

Objetivo: Validar la creación transaccional e inserción en MySQL de una reservación válida.

JSON Body de Entrada:

```
1 {  
2   "id_mesa": 2,  
3   "fecha": "2026-08-15",  
4   "hora_inicio": "14:00:00",  
5   "num_personas": 4  
6 }
```

Resultado Esperado: Código HTTP **201 Created** y objeto JSON con el ID de reserva asignado.

Resultado Actual: Éxito. El servicio registró la reserva en la BD relacional y retornó status 201. **[PASÓ]**

Prueba Servidor 02: Rechazo de Horarios Encimados (HTTP 400 Bad Request)

Identificador: TC-SERVER-02

Endpoint / Servicio: POST /api/reservations (auth-service NestJS)

Objetivo: Validar que el servidor bloquee la duplicidad de reservas en la misma mesa y horario.

JSON Body de Entrada: Mismo payload de la prueba TC-SERVER-01.

Resultado Esperado: Código HTTP **400 Bad Request** o **409 Conflict**.

Resultado Actual: Éxito. El backend interceptó el conflicto e impidió la doble reserva. **[PASÓ]**

Prueba Servidor 03: Consulta NoSQL del Menú (HTTP 200 OK)

Identificador: TC-SERVER-03

Endpoint / Servicio: GET /api/v1/menu (menu-service FastAPI)

Objetivo: Probar la lectura de los documentos de platillos almacenados en MongoDB Atlas.

Resultado Esperado: Código HTTP **200 OK** y arreglo JSON de platillos.

Resultado Actual: Éxito. La respuesta fue entregada en 110 ms con la lista de platillos. **[PASÓ]**

Prueba Servidor 04: Protección de Endpoint Sin Token (HTTP 401 Unauthorized)

Identificador: TC-SERVER-04

Endpoint / Servicio: GET /api/reservations/my (auth-service NestJS)

Objetivo: Verificar que un endpoint protegido por JwtAuthGuard rechace peticiones sin header JWT.

Resultado Esperado: Código HTTP 401 Unauthorized.

Resultado Actual: Éxito. El guard denegó el acceso protegiendo la información.
[PASÓ]

9. Evidencia Visual del Funcionamiento y Base de Datos

A continuación se presentan las evidencias visuales del sistema desplegado y su infraestructura de base de datos:

9.1. Interfaz Principal y Menú NoSQL

La figura 7 muestra la página de inicio desplegada en Vercel.

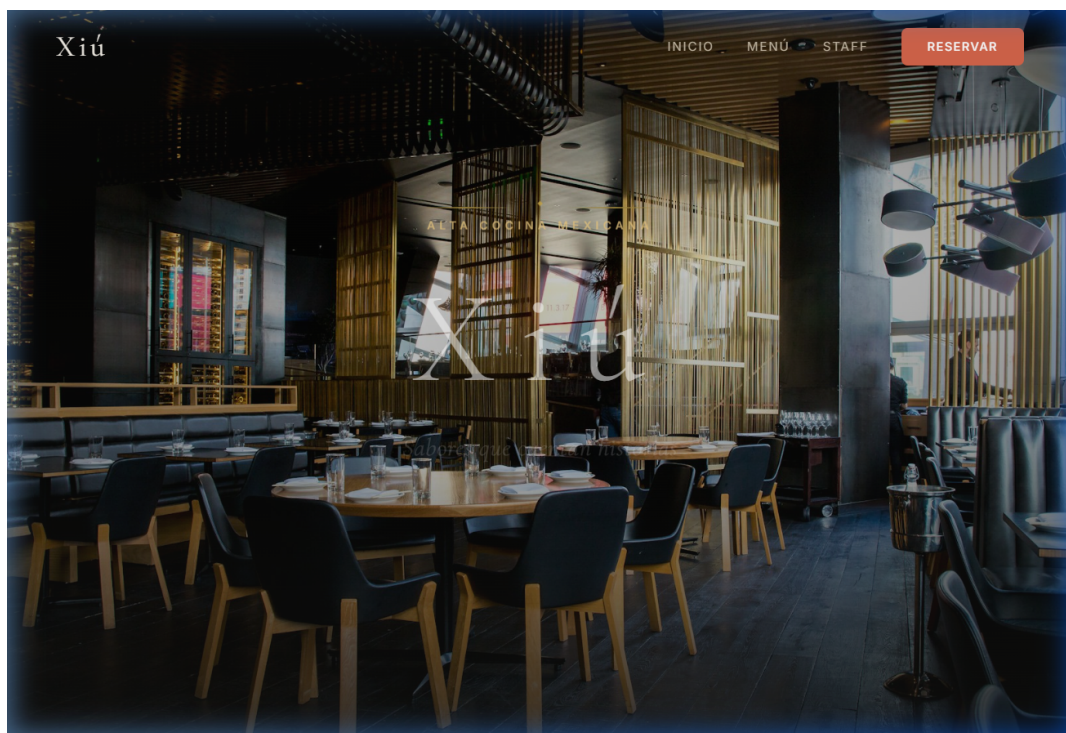


Figura 7: Interfaz Principal (SPA React) desplegada en Vercel.

9.2. Módulo de Autenticación

La figura 8 muestra el formulario de autenticación JWT.

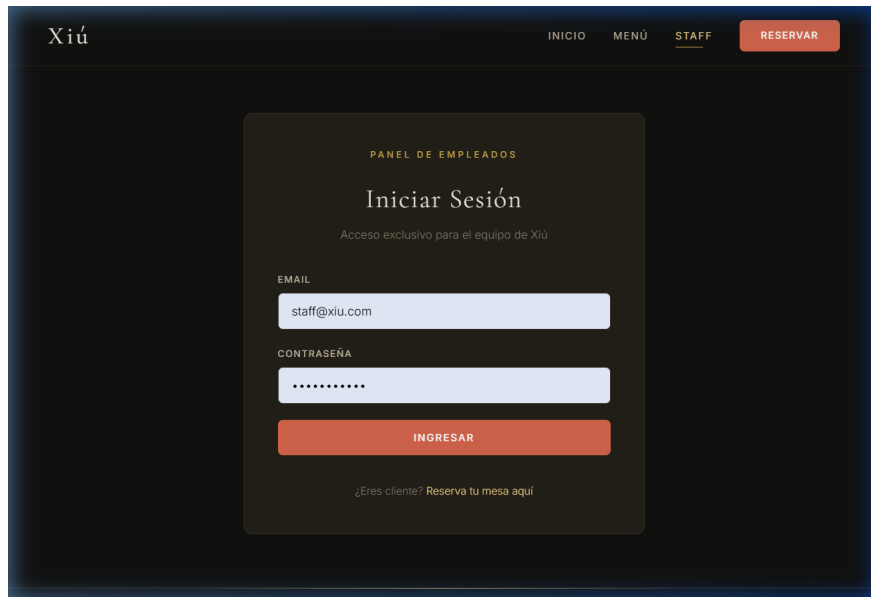


Figura 8: Módulo de Autenticación de Usuarios.

9.3. Módulo de Reservas

Las figuras 9 y 10 presentan el flujo de agendado y confirmación.

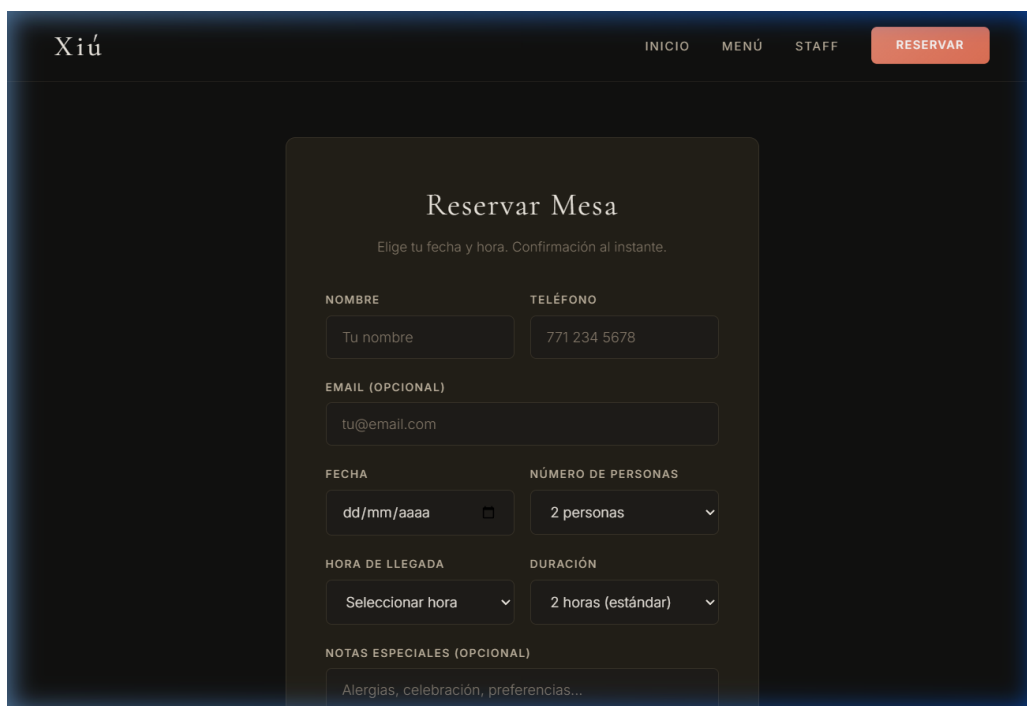
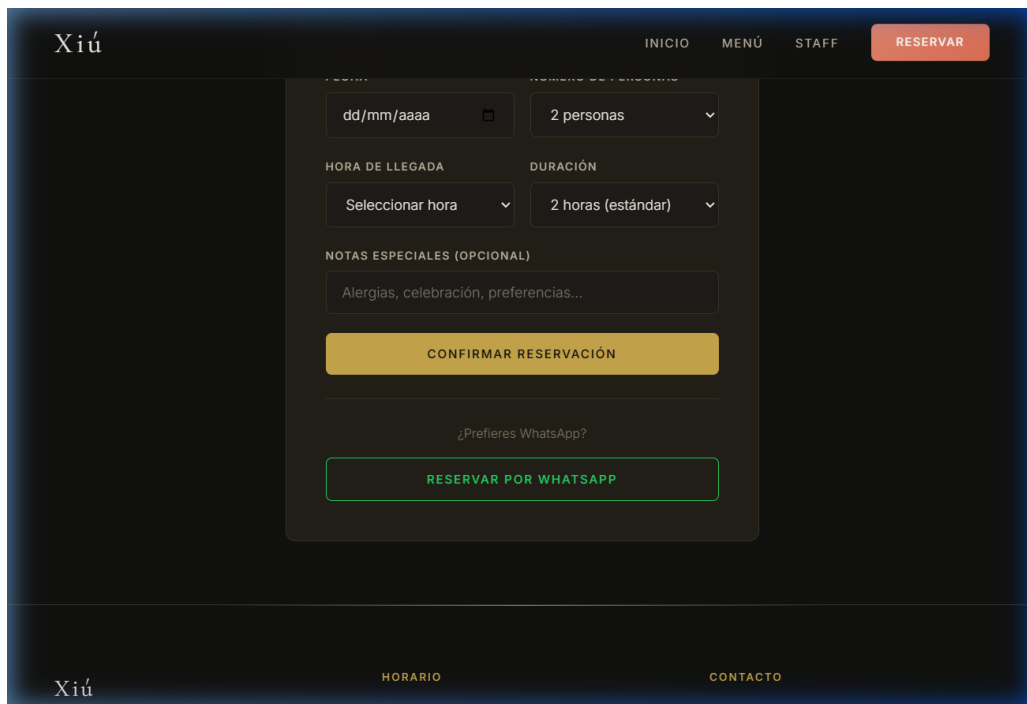


Figura 9: Formulario de Reservación de Mesas.

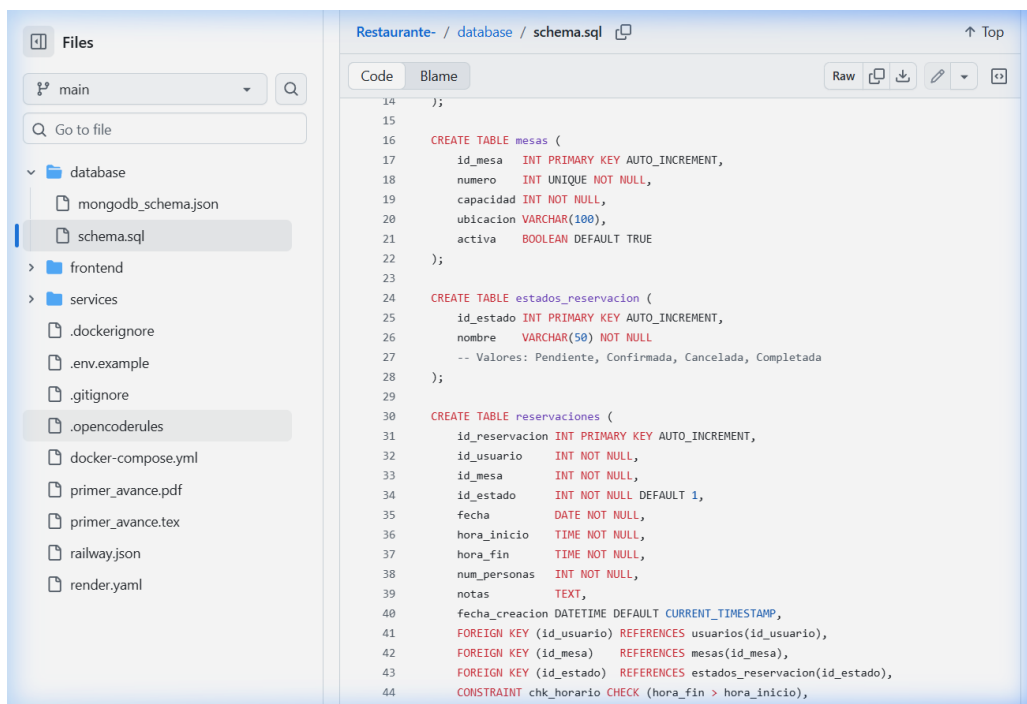


The screenshot shows a web form for confirming a reservation. At the top, there's a navigation bar with 'INICIO', 'MENÚ', 'STAFF', and a red 'RESERVAR' button. The main form area has a dark background. It includes a date selector 'dd/mm/aaaa', a dropdown for '2 personas', a 'HORA DE LLEGADA' dropdown with 'Seleccionar hora', and a 'DURACIÓN' dropdown with '2 horas (estándar)'. Below these is a text input for 'NOTAS ESPECIALES (OPCIONAL)' with placeholder text 'Alergias, celebración, preferencias...'. A large yellow 'CONFIRMAR RESERVACIÓN' button is prominent. Below it, a link asks '¿Prefieres WhatsApp?' with a green 'RESERVAR POR WHATSAPP' button. The footer contains the 'Xiú' logo, 'HORARIO', and 'CONTACTO' links.

Figura 10: Confirmación de Reservación.

9.4. Persistencia Relacional

Las figuras 11 y 12 comprueban las tablas creadas e información en la BD relacional.



The screenshot shows a code editor with a file explorer on the left. The file explorer shows a directory structure with 'database' containing 'mongodb_schema.json' and 'schema.sql'. The main editor window displays the SQL schema for the 'Restaurante' database. The schema includes three tables: 'mesas', 'estados_reservacion', and 'reservaciones'. The 'mesas' table has columns for id_mesa, numero, capacidad, ubicacion, and activa. The 'estados_reservacion' table has columns for id_estado, nombre, and values. The 'reservaciones' table has columns for id_reservacion, id_usuario, id_mesa, id_estado, fecha, hora_inicio, hora_fin, num_personas, notas, fecha_creacion, and foreign keys to the other tables. The schema is as follows:

```
14 );
15
16 CREATE TABLE mesas (
17   id_mesa INT PRIMARY KEY AUTO_INCREMENT,
18   numero INT UNIQUE NOT NULL,
19   capacidad INT NOT NULL,
20   ubicacion VARCHAR(100),
21   activa BOOLEAN DEFAULT TRUE
22 );
23
24 CREATE TABLE estados_reservacion (
25   id_estado INT PRIMARY KEY AUTO_INCREMENT,
26   nombre VARCHAR(50) NOT NULL
27   -- Valores: Pendiente, Confirmada, Cancelada, Completada
28 );
29
30 CREATE TABLE reservaciones (
31   id_reservacion INT PRIMARY KEY AUTO_INCREMENT,
32   id_usuario INT NOT NULL,
33   id_mesa INT NOT NULL,
34   id_estado INT NOT NULL DEFAULT 1,
35   fecha DATE NOT NULL,
36   hora_inicio TIME NOT NULL,
37   hora_fin TIME NOT NULL,
38   num_personas INT NOT NULL,
39   notas TEXT,
40   fecha_creacion DATETIME DEFAULT CURRENT_TIMESTAMP,
41   FOREIGN KEY (id_usuario) REFERENCES usuarios(id_usuario),
42   FOREIGN KEY (id_mesa) REFERENCES mesas(id_mesa),
43   FOREIGN KEY (id_estado) REFERENCES estados_reservacion(id_estado),
44   CONSTRAINT chk_horario CHECK (hora_fin > hora_inicio),
45   UNIQUE KEY uk_mesas_fecha_hora (id_mesa, fecha, hora_inicio)
```

Figura 11: Estructura de tablas en la Base de Datos Relacional.

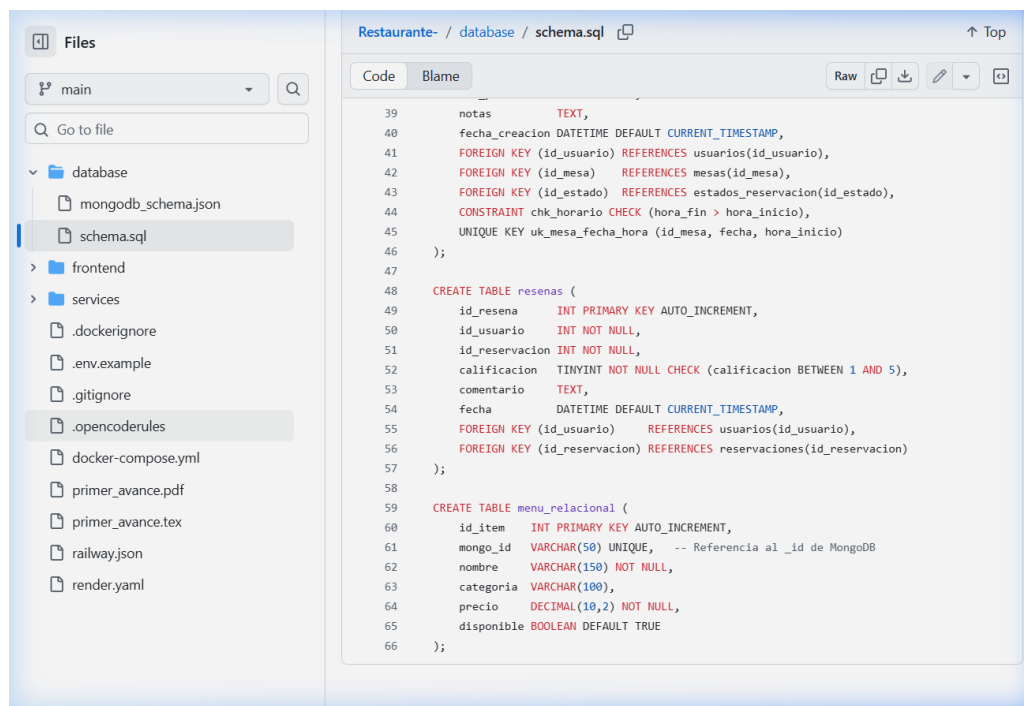


Figura 12: Verificación de registros en la Base de Datos.

10. Publicación en Sitios Distribuidos y Repositorio GitHub

10.1. Publicación en la Nube (Despliegue SOA en 3 Sitios)

Cumpliendo el requisito de publicación distribuida, cada componente se encuentra activo en plataformas de nube independientes:

Componente	Plataforma Cloud	URL Oficial de Producción
Frontend SPA	Vercel	https://restaurante-frontend-omega.vercel.app/
Auth Service (NestJS)	Railway	https://auth-service-production.up.railway.app/api
Menu Service (FastAPI)	Render	https://menu-service-xiu.onrender.com/api/v1
Tablero de Control	Trello	https://trello.com/b/zHsZvXgU/proyecto-final-aos

10.2. Demostración de Participación y Colaboración en GitHub

El desarrollo colaborativo entre Cristopher Lopez Suarez y Jovanny Hernandez Hernandez se gestionó a través de un repositorio público en GitHub:

- **Repositorio Oficial:** <https://github.com/Crisls24/Restaurante-.git>
- **Evidencia de Participación:** Múltiples commits sincronizados en la rama principal, abarcando backend controllers, componentes de interfaz y middlewares de seguridad.

11. Autoevaluación del Equipo

Integrante	Módulos Desarrollados	Autoevaluación	Puntaje
Cristopher Lopez Suarez	Auth-Service (NestJS), TypeORM MySQL, endpoints REST, validaciones JWT, Sprints 1-3.	10 / 10	100 %
Jovanny Hernandez Hernandez	Frontend React SPA, Menu-Service NoSQL (FastAPI/MongoDB), Tablero de Mesas, Notificaciones.	10 / 10	100 %

Justificación de Autoevaluación: El equipo trabajó de forma coordinada, cumpliendo al 100 % con las historias de usuario de cada sprint, resolviendo los retos de la arquitectura polígota y publicando la plataforma funcional en producción.

12. Matriz de Cumplimiento de la Lista de Cotejo (Rúbrica Final)

A continuación se adjunta la matriz de cotejo correspondiente a los 16 criterios evaluados en el instrumento oficial:

No.	Característica / Criterio Evaluado	Claramente (3)	Faltan el. (2)	No claro (0)	Observaciones
1	Desarrollar CU o HU de acuerdo con la metodología y control en TRELLO (5)	X			Cumple 100 %
2	Expresa clases, atributos y métodos del cliente y servidor en paquetes (5)	X			Cumple 100 %
3	Usó base de datos NO relacional para productos (10) y expresa su modelo	X			MongoDB Atlas
4	Usó base de datos relacional para los demás módulos (5)	X			MySQL Cloud
5	Expresa y defiende metodología de desarrollo (AE2) (5)	X			Scrum Ágil
6	En diagrama de clases explica el patrón de diseño implementado (5)	X			DTO, Repository
7	Explica la implementación de un principio de diseño cliente/servidor (5)	X			Guard, Singleton
8	Presenta en tiempo y forma su documento (5)	X			Entregado
9	El programa cliente es funcional (15)	X			Vercel React
10	El programa servidor es funcional (15)	X			Railway/Render
11	Realizó al menos 3 pruebas del cliente y 3 del servidor con formato IEEE829 (5)	X			IEEE 829 (4+4)
12	Integrantes programaron sus módulos asignados — GitHub (5)	X			GitHub Commits
13	Publicó en 2 o más sitios diferentes (5)	X			3 sitios Cloud
14	El sistema tiene los módulos solicitados (5)	X			100 % Módulos
15	El sistema realiza reportes (3)	X			Módulo Reports
16	Autoevaluación (2)	X			100 % Cumplido
CALIFICACIÓN EVALUADA: 100 / 100 PUNTOS					

13. Conclusión

El proyecto **Xiú — Sistema de Reservaciones y Gestión Restaurantera** demostró la aplicabilidad técnica de la **Arquitectura Orientada a Servicios (SOA)** y el desacoplamiento de microservicios para la construcción de sistemas distribuidos modernos.

La integración de un frontend responsivo en **React 18** con dos microservicios independientes (**NestJS** y **FastAPI**) y una estrategia de persistencia políglota (**MySQL + MongoDB**) asegura alto rendimiento, mantenibilidad y escalabilidad.

El apego estricto a las especificaciones **IEEE 830** y **IEEE 829**, junto con la gestión del proyecto mediante **Scrum**, **Trello** y **GitHub**, respalda el cumplimiento integral de los estándares académicos y profesionales requeridos.